Robust HDBSCAN-Based Customer Segmentation and Value Analysis: A Reproducible Framework for Stability-Aware Customer Persona Discovery

Abstract

Customer segmentation supports targeted marketing and customer relationship management, but commonly used partitioning methods require a prespecified number of clusters and can be sensitive to irregular cluster geometry and outliers. This study presents a reproducible, density-based framework for customer persona discovery using Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) on the IBM Telco Customer Churn dataset. The implementation combines column normalization, data-type cleaning, categorical encoding, feature standardization, Principal Component Analysis (PCA), HDBSCAN hyperparameter search, comparisons with K-Means, DBSCAN and Agglomerative Clustering, subsampling-based stability assessment, and post-clustering customer-value and observed-churn profiling. The pipeline evaluates two feature representations—standardized features without PCA and a PCA representation retaining at least 95% cumulative variance—and records internal cluster-quality metrics, cluster sizes and the proportion of observations assigned to noise. Stability is assessed over ten 80% subsamples using the Adjusted Rand Index (ARI), with the noise label included explicitly. The codebase also provides a Streamlit dashboard for exploring the resulting profiles and diagnostics. At the time of drafting, source code and project documentation are available, but the generated experiment-output files have not been provided for independent verification; therefore, numerical findings are intentionally left as placeholders. The paper does not treat cluster-level observed churn rates as predictions of future churn, and it excludes outcome-derived churn and CLTV fields from the primary clustering feature set. The contribution is a leakage-aware and stability-aware experimental workflow that makes segmentation choices auditable and connects cluster structure to descriptive customer-value analysis.

Keywords: HDBSCAN; customer segmentation; Principal Component Analysis; clustering stability; Adjusted Rand Index; customer lifetime value; churn analytics.

1. Introduction

Customer segmentation is a core customer-analytics task: it organizes heterogeneous customers into groups that can be examined and served with differentiated strategies. In telecommunications, customers vary in tenure, service subscriptions, contract arrangements, monthly expenditure and cumulative charges. These differences can matter to customer experience and commercial planning, but a segmentation is only useful when its groups are sufficiently coherent, stable and interpretable to support cautious action.

K-Means is widely used because it is computationally straightforward and produces compact partitions, but it requires the number of clusters to be selected in advance and optimizes a centroid-based objective that can be poorly matched to non-spherical structures. Agglomerative clustering offers a hierarchy but still requires a cut or cluster count for a flat partition. DBSCAN introduced a density-based approach that can identify arbitrarily shaped clusters and noise, while its results can be sensitive to the choice of a single neighbourhood-radius parameter [1]. HDBSCAN extends density-based clustering by examining structure across density levels and selecting stable clusters from a hierarchy [2,3].

Customer records are often represented by mixed numeric and categorical attributes. Encoding and scaling choices affect distance geometry, while redundant dimensions may complicate density estimation. PCA can provide a lower-dimensional representation of standardized numeric inputs, but it changes the geometry on which density-based methods operate; reduced dimensionality is therefore a hypothesis to test, not an assumed improvement [4]. Moreover, a single internal metric cannot establish that clusters are reproducible or commercially meaningful. A segmentation may obtain an attractive silhouette score while assigning many records to noise, fragmenting customers into tiny groups, or changing materially under resampling.

This study develops an auditable pipeline for HDBSCAN-based customer persona discovery using a publicly available IBM Telco Customer Churn dataset. It compares HDBSCAN with and without PCA, explores the effects of `min_cluster_size` and `min_samples`, benchmarks alternative clustering algorithms, evaluates resampling stability with ARI, and summarizes cluster-level tenure, charges, CLTV and observed churn when the chosen dataset variant contains those fields. A Streamlit dashboard provides an interactive interface to inspect the results.

The study makes three practical contributions. First, it packages preprocessing, dimensionality reduction, clustering, evaluation and profile generation in a reproducible workflow. Second, it evaluates stability alongside cluster-quality and noise measures rather than selecting a configuration on one score alone. Third, it separates features used to discover clusters from outcome-derived fields used for post-hoc descriptive analysis. The study is an independent implementation inspired by prior customer-analytics research [8], not a reproduction of that work.

The research questions are: RQ1, can HDBSCAN discover interpretable customer groups without specifying a cluster count beforehand? RQ2, how does PCA change cluster quality, noise assignment and stability? RQ3, how sensitive are results to HDBSCAN parameters? RQ4, how do clusters differ in value-related attributes and observed churn? RQ5, how does HDBSCAN compare with K-Means, DBSCAN and Agglomerative Clustering? Answers to these questions must be based on the completed output files, not on algorithm expectations.

2. Related Work and Literature Review

2.1. Density-based clustering

DBSCAN defines clusters in terms of density reachability and distinguishes core, border and noise observations [1]. It can discover non-convex groups without assigning every observation to a cluster, an advantage when customer profiles contain atypical combinations of attributes. Its radius parameter, however, can be difficult to choose when densities differ across the data. HDBSCAN addresses this limitation by constructing a hierarchy based on mutual-reachability distances and selecting clusters according to stability across density levels [2]. The open-source `hdbscan` implementation makes this family of methods accessible to Python workflows [3]. HDBSCAN does not guarantee that discovered groups are commercially meaningful; interpretation still depends on feature design, parameter settings, data quality and validation.

2.2. Dimensionality reduction and clustering validation

PCA transforms correlated features into orthogonal principal components ordered by the variance they explain. It is useful for summarizing high-dimensional data, but variance preservation is not identical to preservation of local density or cluster boundaries [4]. Accordingly, this project compares clustering on standardized encoded features against clustering on PCA scores retaining at least 95% cumulative variance. The comparison should report both the number of retained components and the cumulative explained variance.

Internal metrics provide complementary but imperfect evidence. The silhouette coefficient compares within-cluster cohesion with the nearest alternative cluster [5]. The Davies–Bouldin index summarizes within-cluster scatter relative to between-cluster separation, while the Calinski–Harabasz index relates between-cluster dispersion to within-cluster dispersion. Each metric is sensitive to assumptions about geometry and is not a direct measure of business utility. For HDBSCAN, the fraction of records assigned label −1 must be reported because a high score calculated only on non-noise records may conceal extensive exclusion.

2.3. Cluster stability and customer analytics

Stability analysis asks whether a clustering solution persists under reasonable perturbations of the data or modelling procedure. The Adjusted Rand Index compares two partitions while adjusting pairwise agreement for chance [6]. In this project, HDBSCAN is refitted on repeated 80% subsamples and its assignments are compared with the full-data labels on the sampled records. Because the implementation includes noise label −1 in the ARI calculation, the result reflects agreement in both cluster membership and noise assignment; it should be interpreted alongside the noise fraction and cluster counts.

Customer lifetime value represents a customer-value construct, but its operational definition depends on available data and the business horizon. The original Telco dataset commonly distributed on Kaggle contains 7,043 records and 21 columns, including tenure, monthly charges, total charges and a churn indicator [7]. An extended dataset variant may include additional fields such as CLTV or churn score; the exact file used must be identified before describing those variables as available. Churn in this dataset records an observed outcome, not a future prediction generated by an unsupervised model. Segment-level churn rates are descriptive summaries and cannot establish that segment membership causes churn.

2.4. Positioning of the present study

The present work does not claim that HDBSCAN is universally superior, nor that PCA necessarily improves segmentation. Its defensible contribution is an implementation-level evaluation protocol that combines a PCA/no-PCA comparison, a defined parameter search, several clustering baselines, subsampling stability and descriptive customer-value profiles in one

documented pipeline. The strength of any empirical conclusion depends on reporting the generated outputs, the dataset variant and the limitations of the chosen evaluation measures.

3. Research Gap and Objectives

The practical gap addressed here is not a claim that previous literature has never combined density-based clustering with customer analytics. Rather, the project operationalizes a transparent set of checks for a specific public telecommunications dataset: (i) compare standardized features with a PCA representation; (ii) expose HDBSCAN parameter sensitivity; (iii) report noise and multiple internal metrics; (iv) quantify repeatability under subsampling; and (v) profile clusters using variables reserved from the clustering input. This design helps readers distinguish a visually attractive segmentation from one that is reproducible and useful for exploratory analysis.

Objective	Operational evidence required	Status at draft time
O1. Discover customer segments	Cluster labels, non-noise cluster count, noise fraction, cluster-size distribution	Code available; output values pending
O2. Compare PCA and no-PCA	PCA component count/variance and metrics for both experiments	Implemented; output values pending
O3. Assess parameter sensitivity	Grid-search results for min_cluster_size × min_samples	Implemented; output values pending
O4. Benchmark algorithms	Metrics for HDBSCAN, K-Means, DBSCAN and Agglomerative	Implemented; output values pending
O5. Evaluate stability	Ten 80% subsamples; ARI mean and standard deviation	Protocol present in code; output values pending
O6. Profile customer value and churn	Cluster-level CLTV/charges/tenure/churn summaries where fields exist	Implemented conditionally; values and dataset schema pending

4. Dataset and Methodology

4.1. Dataset

The implementation is designed to load an IBM Telco Customer Churn CSV whose filename matches common Telco churn naming patterns. The clustering module attempts to import scikit-learn's HDBSCAN first and falls back to the separate `hdbscan` package if needed; the run log reported the scikit-learn implementation, so the final environment record should identify that exact implementation and version. The README documents two candidate sources: the original Kaggle dataset and an extended IBM Telco variant that may contain CLTV, ChurnScore and referral attributes. The original commonly used file has 7,043 customer records and 21 columns [7], but that count must not be transferred automatically to the actual run. The exact file, checksum or version, row/column count, missingness and available variables must be filled from `outputs/dataset_info.json` and the original CSV.

The identifier `CustomerID` is excluded from clustering. The preprocessing module normalizes variations in column names and converts `Tenure`, `MonthlyCharges`, `TotalCharges`, `CLTV`, `ChurnScore` and `NumReferrals` to numeric form where present. The dataset variant is especially important for value analysis: if CLTV is absent, the paper will report charges and tenure as available proxies and will not invent a CLTV measure.

Variable group	Examples in supported schema	Analytical role
Customer identifier	CustomerID	Excluded from clustering; used only for record traceability
Demographics	Gender, SeniorCitizen, Partner, Dependents	Categorical profile features when available
Service subscriptions	PhoneService, InternetService, OnlineSecurity, TechSupport, StreamingTV, etc.	One-hot encoded input features
Account/contract	Tenure, Contract, PaperlessBilling, PaymentMethod	Input features and descriptive profile variables
Financial	MonthlyCharges, TotalCharges	Input features and post-cluster value summaries
Outcome-derived	ChurnLabel, ChurnValue, ChurnScore, CLTV	Excluded from primary clustering features; may be used post hoc if present

4.2. Preprocessing and feature engineering

The preprocessing pipeline first normalizes column names, converts selected numeric fields using numeric coercion and derives a binary churn indicator for post-clustering analysis when a churn label or value is present. Categorical columns listed in the implementation are one-hot encoded. Numeric and encoded features are standardized using `StandardScaler`, so variables with larger raw units do not automatically dominate Euclidean distances. Exact handling of rows that become missing after numeric conversion, as well as any dropped records, must be confirmed from the dataset information generated at runtime.

A leakage-aware design is essential. The source code declares `ChurnLabel`, `ChurnValue`, `ChurnScore` and `CLTV` as leakage-sensitive fields and excludes them from the primary clustering input. ChurnScore is optionally tested in a separate experiment only when present, and the code explicitly warns that it is outcome-derived. That optional comparison should not be presented as the main model. CLTV is used for cluster profiling only when available. This separation prevents outcome-derived information from defining the clusters that are later evaluated against those same outcomes.

4.3. Standardization and PCA

Let $X \in R^{(n \times p)}$ be the encoded feature matrix. Each feature j is standardized as $z_{ij} = (x_{ij} - \mu_j)/\sigma_j$, where μ_j and σ_j are the training-data mean and standard deviation. PCA then projects standardized observations onto a set of orthogonal components. The retained component count K is the smallest count satisfying:

$$K = \min \{ k : \Sigma_{(k \text{ from } j=1)} \lambda_j / \Sigma_{(p \text{ from } j=1)} \lambda_j \geq 0.95 \}$$

Here λ_j denotes the variance associated with component j. The implementation uses a 95% cumulative-variance threshold. Experiment A runs HDBSCAN on the standardized feature matrix directly; Experiment B runs PCA first and then applies the same grid-search procedure in the reduced representation. Since PCA can remove low-variance directions that still contain useful cluster structure, the two representations are treated as competing experimental conditions.

4.4. HDBSCAN formulation

HDBSCAN estimates local density using core distances, defines mutual-reachability distances, constructs a minimum-spanning-tree-based hierarchy and extracts clusters that persist across density levels [2,3]. For observation x_i and a chosen neighbourhood rank k, the core distance is the distance to its k-th nearest neighbour. The mutual-reachability distance between x_i and x_j can be written as:

$$d_{mreach,k}(x_i, x_j) = \max\{core_k(x_i), core_k(x_j), d(x_i, x_j)\}$$

The parameter `min_cluster_size` sets a lower bound on the size of a group considered a cluster, while `min_samples` influences the density conservatism and outlier assignment. Records assigned label −1 are treated as noise/outliers, not as a normal customer persona. This is important for interpretation: noise can represent sparse or unusual profiles, but it can also reflect feature-space limitations or parameter choices.

The implementation searches a grid of `min_cluster_size` and `min_samples` values (the exact grid is defined in `clustering.py`). It selects a candidate using a composite score that starts with silhouette on non-noise records and applies penalties for high noise fractions and excessive cluster fragmentation. Therefore, the selection rule is a project-specific heuristic rather than a universally accepted objective. It should be reported transparently and accompanied by the full grid results.

4.5. Evaluation metrics

The implementation computes the number of non-noise clusters, number and percentage of noise records, cluster-size distribution, silhouette score, Davies–Bouldin index and Calinski–Harabasz index when at least two non-noise clusters make the metrics valid. Silhouette is evaluated on non-noise records only; this must be stated in tables and comparisons. Baseline methods do not necessarily produce noise labels, so their noise percentage and internal metrics are not directly equivalent in all respects.

$$s(i) = [b(i) - a(i)] / \max\{a(i), b(i)\}$$

In this expression, a(i) is the average distance from point i to other points in its own cluster, and b(i) is the smallest average distance from i to another cluster [5]. A higher average silhouette is generally preferable, but it can be affected by cluster geometry and by excluding noise points. The Davies–Bouldin index is generally better when lower; the Calinski–Harabasz index is generally better when higher. None of these measures alone assesses interpretability, stability or commercial benefit.

4.6. Baseline algorithms

The pipeline compares the selected HDBSCAN result with K-Means, DBSCAN and Agglomerative Clustering. K-Means is configured with a cluster count derived from the HDBSCAN result and constrained to a bounded range; Agglomerative Clustering uses the same comparison count. DBSCAN estimates a candidate epsilon from a nearest-neighbour distance percentile and tries a small set of multipliers. These are implementation-specific baselines, not an exhaustive optimization of each algorithm. Any conclusion must acknowledge that the algorithms may have received different degrees of hyperparameter search.

4.7. Subsampling-based stability

The implementation evaluates stability over 10 iterations. In each iteration, 80% of observations are sampled without replacement using a reproducible seed sequence beginning at 42; HDBSCAN is refitted on the sample with the selected `min_cluster_size` and `min_samples`. The subsample labels are compared against the full-data labels restricted to the same sampled indices using the Adjusted Rand Index [6]. The noise label −1 is included as a label in this calculation.

$$\overline{ARI} = (1 / R)\ \Sigma(r=1 \text{ to } R)\ ARI(L_full[I_r],\ L_r)$$

Here R is the number of valid runs, I_r is the index set for subsample r, L_full[I_r] are the full-data labels for those same observations, and L_r are labels obtained after refitting on the subsample. The report should include the mean, standard deviation, individual scores and number of valid runs. Because clustering labels are arbitrary identifiers, ARI is preferable to direct equality of label numbers. A high ARI indicates agreement between partitions, not necessarily a good or actionable segmentation.

4.8. Customer persona and value analysis

After selecting the clustering representation and labels, the pipeline computes per-cluster counts and percentages, mean and median tenure, mean monthly charges, mean total charges, mean CLTV when available, observed churn rate when available, referral summaries and the most common contract, internet-service and payment-method categories. The noise group is reported separately. Persona labels are rule-based summaries generated from observed tenure, monthly charges and observed churn thresholds; they are descriptive labels, not independently validated latent customer types.

Observed churn rate for cluster c is computed as the proportion of records in that cluster whose observed binary churn indicator equals one. This is a retrospective descriptive statistic. It does not estimate future churn probability, and differences between clusters do not imply causality. CLTV is also dataset-dependent; the extended dataset's CLTV field must be described according to its provenance and definition. If unavailable, it must be marked not available rather than reconstructed without a defensible specification.

4.9. System architecture and reproducibility

The codebase is organized into preprocessing, clustering, evaluation, training orchestration and Streamlit presentation modules. `preprocessing.py` handles input discovery and feature preparation; `clustering.py` contains PCA, HDBSCAN, baselines, grid search and metric calculation; `evaluation.py` performs stability analysis, cluster profiling, persona naming and cautious recommendations; `train.py` coordinates experiments and writes JSON/CSV outputs; `app.py` visualizes the saved outputs. The run uses a base random seed of 42 for the stability procedure. Exact Python/library versions and hardware are not currently documented in the available evidence.

[FIGURE 1 PLACEHOLDER — Insert a vector diagram of: CSV dataset → cleaning/encoding → scaling → (A) HDBSCAN and (B) PCA→HDBSCAN → parameter selection → baseline comparison + stability → profiles/outputs → Streamlit dashboard.]

Figure 1. Proposed system architecture. This is a diagram specification, not a measured experimental result.

[FIGURE 2 PLACEHOLDER — Insert end-to-end methodology flowchart with Experiment A, Experiment B, selection rule, comparison, stability and post-hoc profiling.]

Figure 2. End-to-end experimental workflow. Replace this placeholder with a diagram generated from the implemented pipeline.

5. Experimental Setup

The experiment is implemented in Python using pandas, NumPy, scikit-learn, Plotly and Streamlit; the supplied requirements file specifies minimum versions rather than a frozen environment. The README describes the dataset as a CSV placed in a `data/` directory and the output directory as `outputs/`. The training pipeline saves dataset metadata, PCA

metadata, grid-search results, the selected model configuration, algorithm comparisons, stability statistics, cluster profiles, customer assignments and PCA coordinates.

Setting	Implemented or documented value	Submission evidence needed
Random seed	42 for stability subsampling	Record full environment and any other stochastic settings
PCA variance threshold	95% cumulative explained variance	`outputs/pca_info.json`
Stability repetitions	10	`outputs/stability.json`
Stability subsample fraction	0.80 without replacement	`outputs/stability.json`
Primary algorithms	HDBSCAN, K-Means, DBSCAN, Agglomerative	`outputs/algorithm_comparison.json`
HDBSCAN search	Grid search over min_cluster_size and min_samples	`grid_results_A.json`, `grid_results_B.json` and `clustering.py`
Hardware/runtime	Not specified in available files	Record machine, OS and runtimes if discussed
Dataset version/shape	Not verified from run output	`dataset_info.json` and original dataset metadata

The primary result tables should be populated from the generated output files after confirming that the outputs correspond to the final code and dataset version. No numeric result is reported in this draft because those files were not present among the inspected project materials.

6. Results

Preliminary dashboard evidence shared during project development displayed 13 clusters, 8.69% noise points, and a silhouette score of 0.253 for the selected clustering view. These three values are provisional because the underlying best_model.json and output tables were not available for direct inspection; verify them against the final run before treating them as publication-ready. No ARI, PCA component count, algorithm comparison values, or cluster-level profile statistics were available to confirm at this stage.

Preliminary dashboard item	Displayed value (verify against output files)
Number of clusters	13
Noise points	8.69%
Silhouette score	0.253

This section is intentionally prepared as a results-ready manuscript template. The code and README establish which experiments are implemented, but not the empirical values they produced. Each bracketed field below must be replaced with the corresponding output value before submission. Do not retain a placeholder in a final submitted manuscript.

6.1. Dataset and preprocessing summary

Item	Observed value
Dataset filename/version	[INSERT FROM dataset_info.json and source metadata]
Rows before/after cleaning	[INSERT]
Columns in raw dataset	[INSERT]
Features used for clustering	[INSERT n_features and feature_names]
Missing/invalid values and handling	[INSERT from dataset inspection]
PCA components retained	[INSERT from pca_info.json]
Cumulative variance retained	[INSERT from pca_info.json]

The final narrative should report the actual dataset variant and explain any rows excluded or transformed. If the extended version is used, state explicitly which CLTV, churn-score and referral fields are present. The standard 7,043-row version should not be assumed unless the file confirms it.

6.2. PCA versus direct HDBSCAN

Representation	Components	Clusters (excluding noise)	Noise (%)	Silhouette*	DBI*	CH*	Runtime (s)
Standardized features (no PCA)	p = [INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
PCA (≥95% variance)	K = [INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]

*Internal metrics are computed on non-noise records by the current implementation. Add the exact selection criterion and ensure metric comparisons use clearly stated feature spaces. Interpret PCA as beneficial only if the observed trade-offs support that conclusion.

[FIGURE 3 PLACEHOLDER — PCA cumulative explained-variance curve from pca_model.explained_variance_ratio_.]

Figure 3. PCA explained variance. Generate from the fitted PCA object or stored component variance ratios.

6.3. HDBSCAN hyperparameter sensitivity

Report how cluster count, noise fraction, silhouette, Davies–Bouldin index and Calinski–Harabasz index vary across `min_cluster_size` and `min_samples`. The selected configuration is chosen by the implemented composite heuristic, which penalizes high noise and excessive fragmentation; it should not be described as a global optimum.

Representation	min_cluster_size	min_samples	Clusters	Noise (%)	Silhouette	Composite selection score
No PCA	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
PCA	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]

[FIGURE 4 PLACEHOLDER — Heatmap or faceted plot of hyperparameter results from grid_results_A.json and grid_results_B.json. Failed/invalid runs should be shown separately.]

Figure 4. HDBSCAN parameter sensitivity. Values must be generated from the actual grid-search outputs.

6.4. Algorithm comparison

Algorithm	Clusters	Noise (%)	Silhouette	Davies–Bouldin	Calinski–Harabasz	Runtime (s)
HDBSCAN	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
K-Means	[INSERT]	Not applicable / 0 by construction	[INSERT]	[INSERT]	[INSERT]	[INSERT]
DBSCAN	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
Agglomerative	[INSERT]	Not applicable / 0 by construction	[INSERT]	[INSERT]	[INSERT]	[INSERT]

Baseline comparisons must acknowledge the different parameter-search effort and the fact that K-Means and Agglomerative Clustering assign every record to a cluster, while density-based methods can designate noise. No algorithm should be declared superior until the metrics and interpretability trade-offs have been examined.

[FIGURE 5 PLACEHOLDER — Algorithm comparison chart with separate panels for silhouette, DBI, CH, cluster count and noise fraction.]

Figure 5. Comparison of clustering algorithms using the completed experiment outputs.

6.5. Cluster stability

Stability setting/result	Value
Subsample fraction	0.80
Requested iterations	10
Valid runs	[INSERT]

Stability setting/result	Value
Mean ARI	[INSERT]
Standard deviation of ARI	[INSERT]
Individual ARI values	[INSERT]
Noise treatment	Noise label −1 included in ARI

[FIGURE 6 PLACEHOLDER — Distribution/strip plot of individual ARI scores from stability.json, with mean and standard deviation annotated.]

Figure 6. Subsampling stability. Interpret the ARI in conjunction with cluster counts and noise fractions; a single mean score is insufficient.

6.6. Customer persona profiles and value analysis

Cluster/persona	Count (%)	Mean tenure	Mean monthly charges	Mean total charges	Mean CLTV (if available)	Observed churn (%)
Cluster / Noise [INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT or N/A]	[INSERT or N/A]
Cluster / Noise [INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT or N/A]	[INSERT or N/A]
Cluster / Noise [INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT or N/A]	[INSERT or N/A]

The final profile table must contain one row for every discovered cluster and a separate row for noise when present. Use observed means/medians and category modes from `cluster_profiles.csv`; do not invent labels such as “loyal” or “high-value” unless the corresponding summaries justify them. If CLTV is absent, report it as unavailable and do not infer it from total charges without a defined valuation model.

[FIGURE 7 PLACEHOLDER — Cluster size distribution, including noise as a separate category.]

Figure 7. Cluster-size distribution.

[FIGURE 8 PLACEHOLDER — CLTV by cluster, only if CLTV exists in the selected dataset. Include distribution or uncertainty, not just means.]

Figure 8. CLTV distribution by cluster, if supported by the dataset.

[FIGURE 9 PLACEHOLDER — Observed churn rate by cluster, with denominators/counts. This is retrospective description, not a prediction.]

Figure 9. Observed churn rate by cluster.

6.7. Summary of findings

Research question	Evidence to report	Finding
RQ1: Are personas discoverable?	Cluster count, sizes, noise and profile separation	[COMPLETE AFTER RESULTS]
RQ2: Does PCA change outcomes?	PCA/no-PCA metrics and retained variance	[COMPLETE AFTER RESULTS]
RQ3: Are results parameter-sensitive?	Grid search heatmaps and selected settings	[COMPLETE AFTER RESULTS]
RQ4: Do clusters differ in value/churn?	Profile table and observed rate/value comparisons	[COMPLETE AFTER RESULTS]
RQ5: How do baselines compare?	Metrics, noise, runtime and interpretability	[COMPLETE AFTER RESULTS]

7. Discussion

The discussion should be completed after the result placeholders are replaced. It should answer each research question using the measured evidence and separate observations from explanations. If PCA improves an internal metric but increases noise or reduces stability, that trade-off should be reported rather than summarized as a simple improvement. If HDBSCAN yields

a high noise fraction, the result may indicate meaningful outlier detection, a mismatch between the feature representation and density assumptions, or overly conservative settings; it should not automatically be framed as a success.

A comparison with K-Means, DBSCAN and Agglomerative Clustering must consider that each algorithm defines clusters differently and may have received different search effort. Silhouette, Davies–Bouldin and Calinski–Harabasz scores are internal geometric criteria, not external ground truth. The HDBSCAN stability analysis provides a separate measure of repeatability, but the ARI may be influenced by the proportion of noise points because noise is treated as a shared label. Report individual ARI values and inspect whether the same broad profiles persist across runs.

Business interpretation should follow the cluster summaries. For example, a segment with shorter tenure and higher observed churn may motivate closer study of onboarding or contract preferences, but the descriptive relationship does not establish a causal mechanism. A high-spending group may be commercially important, but monthly charges are not equivalent to profit. Recommendations should therefore be framed as hypotheses for further testing, not claims of revenue improvement or churn reduction.

8. Business Implications

The dashboard is intended to help an analyst inspect segment sizes, distributions, hyperparameter experiments, algorithm comparisons, stability and customer profiles. A segment with elevated observed churn may warrant a targeted retention hypothesis; a long-tenure group with lower observed churn may be suitable for loyalty research; and noise points may merit separate inspection rather than forced assignment. Each action should be validated using operational data, stakeholder review and—where possible—controlled campaign evaluation.

The framework does not itself execute campaigns, estimate causal effects or forecast future churn. Its business value is exploratory: it provides a structured way to generate, compare and audit hypotheses about customer heterogeneity. Commercial impact would require a subsequent evaluation of interventions, costs, retention outcomes and profitability.

9. Limitations

Several limitations constrain interpretation. First, the analysis uses one public telecommunications dataset and may not generalize to other markets, customer populations or time periods. Second, the exact dataset variant determines whether CLTV, ChurnScore and referral fields exist; dataset versioning is required for reproducibility. Third, one-hot encoding and standardization define a particular distance geometry for mixed customer attributes. Alternative encodings or feature selections may produce different clusters. Fourth, PCA preserves variance rather than cluster structure and can suppress low-variance but discriminative directions.

Fifth, HDBSCAN is sensitive to `min_cluster_size`, `min_samples`, distance metric and input representation. The current selection heuristic penalizes noise and fragmentation using hand-set thresholds, so its preference structure is a design choice. Sixth, the baseline algorithms are not exhaustively tuned in the supplied implementation, limiting fairness of comparisons. Seventh, internal validation metrics have geometric biases and are calculated on non-noise observations in the current implementation. Eighth, the stability test uses ten subsamples and includes noise as a label; this is useful but not exhaustive evidence of stability. Ninth, rule-based persona names can oversimplify continuous differences. Finally, observed churn and available CLTV fields are retrospective/descriptive and do not prove future outcomes or causal business impact.

10. Conclusion and Future Work

This paper describes a reproducible HDBSCAN-based customer segmentation pipeline for the IBM Telco Customer Churn dataset. The implementation combines preprocessing, standardization, PCA and direct-feature experiments, HDBSCAN parameter search, comparisons with K-Means, DBSCAN and Agglomerative Clustering, subsampling-based ARI stability assessment, post-hoc persona profiling and a Streamlit dashboard. It explicitly separates outcome-derived churn/CLTV fields from the primary clustering input and treats noise observations as a meaningful part of the analysis.

Empirical conclusions remain pending because the generated JSON and CSV result files were not available in the inspected project materials. Consequently, this draft makes no claim that HDBSCAN or PCA outperforms the baselines and reports no fabricated cluster counts, metric values or customer-value findings. Before submission, the result tables must be populated from a final run and the discussion rewritten to reflect those results. Future work could test additional datasets, alternative mixed-data distances, broader baseline tuning, more stability perturbations, temporal segmentation and a separately validated supervised churn model. Any future predictive or optimization component should be described as new work, not as part of the present implementation.

List of Figures

- Figure 1. Proposed system architecture (placeholder; draw from actual code modules).
- Figure 2. End-to-end methodology flowchart (placeholder).
- Figure 3. PCA cumulative explained-variance curve (generate from pca_info.json or fitted PCA).
- Figure 4. HDBSCAN hyperparameter sensitivity for direct and PCA representations.
- Figure 5. Clustering algorithm comparison across internal metrics, cluster count and noise fraction.
- Figure 6. Subsampling ARI distribution with mean and standard deviation.
- Figure 7. Cluster-size distribution, including noise.
- Figure 8. CLTV distribution by cluster, only if CLTV is present.
- Figure 9. Observed churn rate by cluster, with denominators.

List of Tables

- Table 1. Research objectives and evidence status.
- Table 2. Dataset variable groups and analytical roles.
- Table 3. Experimental settings and required evidence.
- Table 4. Dataset and preprocessing summary.
- Table 5. PCA versus no-PCA results.
- Table 6. HDBSCAN hyperparameter sensitivity.
- Table 7. Algorithm comparison.
- Table 8. Cluster stability results.
- Table 9. Customer persona profiles and value analysis.
- Table 10. Research questions and findings.

References

[1] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD), 1996, pp. 226–231. Available: https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf

[2] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, “Hierarchical density estimates for data clustering, visualization, and outlier detection,” ACM Trans. Knowl. Discov. Data, vol. 10, no. 1, Art. no. 5, pp. 1–51, 2015, doi: 10.1145/2733381.

[3] L. McInnes, J. Healy, and S. Astels, “hdbscan: Hierarchical density based clustering,” J. Open Source Softw., vol. 2, no. 11, Art. no. 205, 2017, doi: 10.21105/joss.00205.

[4] I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” Philos. Trans. R. Soc. A, vol. 374, no. 2065, Art. no. 20150202, 2016, doi: 10.1098/rsta.2015.0202.

[5] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” J. Comput. Appl. Math., vol. 20, pp. 53–65, 1987, doi: 10.1016/0377-0427(87)90125-7.

[6] L. Hubert and P. Arabie, “Comparing partitions,” J. Classification, vol. 2, pp. 193–218, 1985, doi: 10.1007/BF01908075.

[7] Blastchar, “Telco Customer Churn,” Kaggle dataset, commonly distributed as WA_Fn-UseC_-Telco-Customer-Churn.csv. Available: https://www.kaggle.com/datasets/blastchar/telco-customer-churn. Accessed: Oct. 27, 2026. Verify the exact version used in the project before submission.

[8] M. Elveny, M. K. M. Nasution, U. R. P. Nasution, L. Silviana, and T. S. M. T. Wook, “An intelligent multi-objective analytics framework for customer segmentation and value-based decision-making,” Decision Analytics Journal, vol. 19, Art. no. 100699, 2026, doi: 10.1016/j.dajour.2026.100699. Used as inspiration and structural reference, not as evidence for this implementation.